SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as “analyzing, “analysis”, and “analytical”. The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.

2. An AI-based sentiment analysis platform receives product reviews containing words like “disagree”, “agreement”, and “agreeable”. The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as “govern”, “government”, and “governance”. The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.

4.A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.
Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers.
Analyze how prefix addition and suffix transformation affect meaning and word class.
Apply the process to decompose each word and map them to a unified base representation.
Derive the morphological structure and normalized root for each input word.

- Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.
Identify suffix patterns and map all variations to a common base form without altering meaning.
Apply the process to extract root forms and classify each transformation.
Determine the morphological breakdown and normalized output for each word.

- Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Assessment Tool 2 Completed

Question 1 Morphological normalization for search indexing

Answer

The three forms belong to one lexical family, but they do not share a single productive English suffix pattern. Analyzing is the present participle of analyze. Analysis is a related noun whose borrowed formation is not safely recovered by chopping a suffix from analyze. Analytical is formed from analytic with -al. The index therefore keeps each lemma and assigns a shared family key.

Code

```
records = {
    "analyzing": ("analyze", "-ing", "inflectional participle", "analysis"),
    "analysis": ("analysis", "none", "related derivational noun", "analysis"),
    "analytical": ("analytic", "-al", "derivational adjective", "analysis"),
}

print("word       root       affix   type                        index key")
for word, (root, affix, kind, key) in records.items():
    print(f"{word:10} {root:10} {affix:7} {kind:26} {key}")
```

Executed output

```
word       root       affix   type                       index key
analyzing  analyze    -ing    inflectional participle     analysis
analysis   analysis   none    related derivational noun   analysis
analytical analytic   -al     derivational adjective      analysis
```

The shared key is an indexing family label, not a claim that all three surface forms have the same grammatical lemma. Keeping the original word and its lemma avoids losing the noun/adjective/verb distinction.

Question 2 Parsing agree word forms

Answer

Disagree contains the prefix dis- and the base agree. Agreement uses the noun-forming suffix -ment, while agreeable uses the adjective-forming suffix -able. Derivation changes meaning or word class. Disagree expresses non-agreement; agreement names a state or result; agreeable describes something pleasant or willing to agree. Sentiment still depends on sentence context.

Code

```
def parse(word):
    prefix = ""
    rest = word
    if rest.startswith("dis"):
        prefix, rest = "dis-", rest[3:]

    suffix = ""
    for ending in ("ment", "able"):
        if rest.endswith(ending):
            suffix = "-" + ending
            rest = rest[:-len(ending)]
            break

    kind = "derivational" if prefix or suffix else "base"
    return prefix or "none", rest, suffix or "none", kind

meaning = {
    "disagree": "not share the same view; polarity depends on context",
    "agreement": "state or result of agreeing; often neutral or positive",
    "agreeable": "pleasant or willing to agree; often positive",
}

for word in ("disagree", "agreement", "agreeable"):
    prefix, root, suffix, kind = parse(word)
    print(word, prefix, root, suffix, kind, meaning[word], "normalize=agree")
```

Executed output

```
disagree dis- agree none derivational not share the same view; polarity depends on context normalize=agree
agreement none agree -ment derivational state or result of agreeing; often neutral or positive normalize=agree
agreeable none agree -able derivational pleasant or willing to agree; often positive normalize=agree
```

The prefix is not itself a reliable sentiment label. A review such as “I disagree that the product is bad” can be positive despite containing disagree.

Question 3 Normalizing govern word forms

Answer

Government is govern plus the noun-forming suffix -ment. Governance is govern plus -ance. Each has one derivational layer from the verb govern, but their meanings differ: government usually refers to an institution or governing body, while governance refers to the process or system of governing. The normalized family key supports topic grouping while the original form preserves the distinction.

Code

```
forms = {
    "govern": ("govern", "none", 0, "verb; direct or control"),
    "government": ("govern", "-ment", 1, "noun; governing body or system"),
    "governance": ("govern", "-ance", 1, "noun; process or method of governing"),
}

for word, (root, suffix, level, meaning) in forms.items():
    print(f"{word:12} root={root:7} suffix={suffix:6} "
          f"level={level} normalized=govern meaning={meaning}")
```

Executed output

```
govern       root=govern  suffix=none   level=0 normalized=govern meaning=verb; direct or control
government   root=govern  suffix=-ment  level=1 normalized=govern meaning=noun; governing body or system
governance   root=govern  suffix=-ance  level=1 normalized=govern meaning=noun; process or method of governing
```

The suffixes are derivational because they form nouns from the verb. For high-quality clustering, store both the family key and the surface word so the distinct senses remain available.

Question 4 Parsing activate word forms

Answer

The derivational chain is act to active to activate to activation. Reactivation adds re- to activation, meaning that activation happens again. Active and activate change word class along the chain; -ion forms the noun activation. Both activation and reactivation are nouns, with re- adding a repeated-action meaning.

Code

```
analysis = {
    "activate": {
        "prefix": "none", "root": "act", "suffixes": "-ive + -ate",
        "sequence": "act -> active -> activate", "class": "verb",
        "meaning": "cause something to become active"
    },
    "activation": {
        "prefix": "none", "root": "act", "suffixes": "-ive + -ate + -ion",
        "sequence": "act -> active -> activate -> activation", "class": "noun",
        "meaning": "process or result of activating"
    },
    "reactivation": {
        "prefix": "re-", "root": "act", "suffixes": "-ive + -ate + -ion",
        "sequence": "re- + activation", "class": "noun",
        "meaning": "activating again"
    },
}

for word, row in analysis.items():
    print(word, row["prefix"], row["root"], row["suffixes"],
          row["sequence"], row["class"], "normalize=act", row["meaning"])
```

Executed output

```
activate none act -ive + -ate act -> active -> activate verb normalize=act cause something to become active
activation none act -ive + -ate + -ion act -> active -> activate -> activation noun normalize=act process or
result of activating
reactivation re- act -ive + -ate + -ion re- + activation noun normalize=act activating again
```

The decomposition is a teaching analysis of the requested family. A production analyzer would consult a lexicon because not every English word can be reliably split by suffix rules alone.

Question 5 Inflectional normalization of create

Answer

Creates is the third-person singular present form, and creating is the present participle or progressive form. These are inflectional changes: they preserve the verb lemma create. The final e is retained in creates and dropped before -ing in creating.

Code

```
def parse_verb(word):
    if word == "create":
        return "", "create", "none", "base form", "create"
    if word == "creates":
        return "", "create", "-s", "third-person singular present", "create"
    if word == "creating":
        return "", "create", "-ing (drop final e)", "present participle", "create"
    raise ValueError("This teaching parser covers the requested forms.")

for word in ("create", "creates", "creating"):
    prefix, root, suffix, feature, normalized = parse_verb(word)
    print(word, prefix or "none", root, suffix, feature, normalized)
```

Executed output

```
create none create none base form create
creates none create -s third-person singular present create
creating none create -ing (drop final e) present participle create
```

The example parser is intentionally limited to the three requested forms; a general-purpose system needs more spelling rules and a verb lexicon.